



INSTITUTO POLITÉCNICO NACIONAL

UPIICSA

ABSTRACCIÓN Y USO DE DATOS

EQUIPO: SYNTAX ERROR

3CM32

REPORTE PRÁCTICA 7 BIS

Desarrollo: Manejo de una lista de manera dinámica por medio de punteros al mismo tipo de dato de la pila, considerando la referencia (lista enlazada dinámica).

Errores:

- Error 1: Pérdida de la referencia al inicio de la lista (*perder la cabeza*) al reasignar el puntero principal sin conectar primero el nuevo nodo.
- Error 2: Violación de segmento por intentar leer el campo de un nodo posterior a haber liberado su memoria con delete.

Soluciones:

- Solución 1: Conectar el nuevo nodo al resto de la estructura antes de actualizar el puntero de la cabeza (nuevo->siguiente = cabeza; cabeza = nuevo;).
- Solución 2: Guardar una referencia auxiliar al siguiente nodo antes de liberar el elemento actual durante los recorridos de limpieza o borrado.

Código:

```
#include <iostream>
#include <string>
#include <limits>
using namespace std;

class Nodo {
private:
    int dato;
    Nodo* siguiente;

public:
    Nodo(int valor, Nodo* sig = nullptr) {
        dato = valor;
        siguiente = sig;
    }

    int getDato() const {
        return dato;
    }

    Nodo* getSiguiente() const {
```

```
        return siguiente;
    }

    void setSiguiente(Nodo* sig) {
        siguiente = sig;
    }
};

class Pila {
private:
    Nodo* tope;
    int cantidad;

public:
    Pila() {
        tope = nullptr;
        cantidad = 0;
    }

    ~Pila() {
        vaciar();
    }

    bool estaVacia() const {
        return tope == nullptr;
    }

    int tamanio() const {
        return cantidad;
    }

    bool push(int valor) {
        Nodo* nuevo = new (nothrow) Nodo(valor, tope);
        if (nuevo == nullptr) return false;
        tope = nuevo;
        cantidad++;
        return true;
    }

    bool pop(int& valor) {
        if (estaVacia()) return false;
        Nodo* aux = tope;
        valor = aux->getDato();
        tope = aux->getSiguiente();
        delete aux;
        cantidad--;
        return true;
    }
```

```
    bool cima(int& valor) const {
        if (estaVacia()) return false;
        valor = tope->getDato();
        return true;
    }

    void vaciar() {
        int descartado;
        while (pop(descartado)) {
        }
    }

    void mostrar() const {
        if (estaVacia()) {
            cout << "La pila esta vacia." << endl;
            return;
        }
        cout << "\nCima" << endl;
        Nodo* actual = tope;
        int posicion = cantidad - 1;
        while (actual != nullptr) {
            cout << "  [" << posicion << "] " << actual->getDato() << endl;
            actual = actual->getSiguiente();
            posicion--;
        }
        cout << "Base" << endl;
    }
};

int leerEntero(const string& mensaje) {
    int valor;
    while (true) {
        cout << mensaje;
        if (cin >> valor) return valor;
        cout << "Entrada no valida, intente de nuevo." << endl;
        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
    }
}

int main() {
    Pila pila;
    int opcion;

    do {
        cout << "\nPila Dinamica" << endl;
        cout << "1. Insertar (push)" << endl;
```

```
        cout << "2. Eliminar (pop)" << endl;
        cout << "3. Ver cima" << endl;
        cout << "4. Mostrar pila" << endl;
        cout << "5. Tamanio actual" << endl;
        cout << "6. Vaciar pila" << endl;
        cout << "0. Salir" << endl;
        opcion = leerEntero("Opcion: ");

        switch (opcion) {
            case 1: {
                int valor = leerEntero("Ingrese el valor: ");
                if (pila.push(valor))
                    cout << "Valor " << valor << " insertado." << endl;
                else
                    cout << "No hay memoria disponible." << endl;
                break;
            }
            case 2: {
                int valor;
                if (pila.pop(valor))
                    cout << "Elemento eliminado: " << valor << endl;
                else
                    cout << "Subdesbordamiento: la pila esta vacia." << endl;
                break;
            }
            case 3: {
                int valor;
                if (pila.cima(valor))
                    cout << "Elemento en la cima: " << valor << endl;
                else
                    cout << "La pila esta vacia." << endl;
                break;
            }
            case 4:
                pila.mostrar();
                break;
            case 5:
                cout << "Elementos en la pila: " << pila.tamanio() << endl;
                break;
            case 6:
                pila.vaciar();
                cout << "Pila vaciada." << endl;
                break;
            case 0:
                cout << "Hasta luego." << endl;
                break;
            default:
                cout << "Opcion no valida." << endl;
```

```
        }
    } while (opcion != 0);

    return 0;
}
break;
            case 6:
                pila.vaciar();
                cout << "Pila vaciada." << endl;
                break;
            case 0:
                cout << "Hasta luego." << endl;
                break;
            default:
                cout << "Opcion no valida." << endl;
        }
    } while (opcion != 0);

    return 0;
}
```